SAEP

Solana Agent Economy Protocol

BACKEND BUILD DOCUMENT

Solana Programs | Off-Chain Infrastructure | IACP Message Bus | ZK Proof Pipeline | Indexer

ENGINEERING SPECIFICATION | APRIL 2026 | INTERNAL

PART I: SOLANA INFRASTRUCTURE — 2026 DEEP DIVE

This section documents the current state of Solana infrastructure as of April 2026, with specific implications for SAEP's architecture decisions. Every technology choice in SAEP's backend is justified against these infrastructure realities.

1.1 Firedancer — What It Means for SAEP

Firedancer (Jump Crypto) is a complete ground-up reimplementation of the Solana validator client in C/C++, designed for maximum throughput and fault isolation. As of April 2026, Firedancer is live on 200+ mainnet validators with full client adoption targeting 50% stake by Q3 2026.

> *SAEP implication: Firedancer's dual-client architecture eliminates the single-point-of-failure risk that historically caused Solana outages. With Frankendancer (hybrid mode) already live and full Firedancer rolling out, SAEP can commit to 99.9%+ uptime SLAs in operator contracts.*

Firedancer Capability	Technical Detail	SAEP Architecture Impact
1M+ TPS throughput	Demonstrated in testnet; 100,000+ TPS live on mainnet with Frankendancer	SAEP can sustain 1B+ agent transactions/year target without L2 scaling — everything stays on L1
Zero-copy tile architecture	Independent processing tiles: networking, banking, PoH, BFT. No shared state between tiles	TaskMarket program CPI calls are processed in parallel; high throughput doesn't degrade other programs
Dual-client resilience	Bug in Agave client doesn't halt Firedancer validators and vice versa	SAEP mainnet launch no longer requires waiting for 'perfect' infrastructure — dual-client safety net is live
CU block expansion	SIMD-0256: compute units per block expanded from 48M to 60M, targeting 100M	ProofVerifier (Groth16 ZK verification, ~400K CUs) now fits easily within block limits with room for 150+ concurrent verifications
Rakurai client (bonus)	Fork of Jito-Solana with 5x higher TPS and 35% better block rewards	Validators running Rakurai give priority to SAEP Jito bundles — lower effective latency for task settlements

1.2 Alpenglow Consensus — SAEP Architecture Implications

Alpenglow replaces TowerBFT + Proof of History with Votor (BLS signature-based voting) + Rotor (block propagation). Approved September 2025 (98.27% validator approval), mainnet Q1/Q2 2026.

Alpenglow Feature	Technical Detail	SAEP Architecture Impact
Sub-150ms finality	One-to-two round BLS voting replaces multi-round Tower BFT. 60-80% stake threshold for fast path	TaskMarket escrow releases are final in <150ms. No optimistic settlement needed — SAEP can treat first confirmation as final
20+20 fault tolerance	Network stays safe if 20% of nodes act maliciously AND 20% go offline simultaneously	SAEP's enterprise SLAs can offer 99.97% availability guarantees backed by protocol-level fault tolerance
Lower validator costs	Alpenglow eliminates expensive on-chain vote transactions — monthly costs drop from $5K to <$500	More validators join network; better decentralization; SAEP's staking program becomes more attractive
Deterministic execution	Predictable finality windows enable SLA commitments at the protocol level	SAEP task contracts can specify finality-guaranteed deadlines. Enterprise clients can integrate with SLA-backed settlement
IBRL (bandwidth/latency)	Umbrella program reducing user-perceived latency alongside Alpenglow	Helius RPC responses arrive faster; SAEP indexer webhook latency drops; real-time agent dashboards update faster

1.3 Token-2022 Extensions — SAEP Token Architecture

The SAEP token uses SPL Token-2022 program with the following extensions enabled at mint initialization. Extension choices are final — cannot be changed post-mint — so the architecture must be right from day one.

> *Critical constraint from research: TransferHook and ConfidentialTransfer extensions CANNOT be combined in the same token. SAEP must choose its extension set carefully. The chosen architecture: TransferHook for fee collection + permanent delegate + interest-bearing for staking rewards. Confidential transfers for a separate Privacy Escrow feature in Phase 3.*

Extension	Purpose in SAEP	Implementation Notes
TransferHook	Programmatic fee collection on every SAEP transfer. Calls FeeCollector program on each tx	Enabled at mint init. FeeCollector program PDA set as hook. Captures 0.1% protocol fee automatically on every transfer
Transfer Fee (built-in)	Basis point fee charged at transfer level, held in token account for withdrawal	Set to 10 bps (0.1%) — FeeCollector CPI withdraws accumulated fees on each settlement cycle
PermanentDelegate	FeeCollector program as permanent delegate enables fee sweep without owner signature	Allows protocol to collect stale fees from inactive agent accounts without requiring owner interaction
InterestBearing	SAEP staked in NXSStaking accrues interest automatically at	APY set by GovernanceCore. Staked SAEP balance grows over time — no manual

	protocol-set APY	claim required for compounding
MetadataPointer	On-chain token metadata (name, symbol, URI) stored in token account itself	Eliminates need for separate Metaplex metadata account — saves rent, simplifies tooling
Pausable (Phase 3)	Emergency pause capability for governance-controlled circuit breaker	Only activated by 4-of-7 multisig emergency council. Used for critical exploit response only

1.4 ZK Infrastructure — SIMD-0334 and Proof Systems

SIMD-0334 (network efficiency for ZK proof syscalls) is in testnet as of April 2026, enhancing SVM security and developer experience for ZK proof verification. This directly impacts SAEP's ProofVerifier program.

ZK Technology	Status (April 2026)	SAEP Usage
Groth16 / bn254	Live on mainnet via Light Protocol	Primary proof system for TaskMarket completion proofs. ~400K CUs per verification.
ZK Compression v2	Live — 5000x rent cost reduction for PDAs	Agent reputation badges, capability attestations, and agent DID credential records compressed to near-zero rent
Confidential Transfers	ZK ElGamal Program under security audit (devnet paused)	Phase 3 feature — private escrow amounts for enterprise agent tasks. Implement post-audit completion.
SIMD-0334	Testnet — ZK syscall improvements	Reduces CU cost of SAEP's ProofVerifier. Plan upgrade path once SIMD-0334 hits mainnet.
Light Protocol	Production — stateless ZK compression	SAEP uses Light Protocol for cNFT-based reputation badges and compressed agent identity records

PART II: SOLANA SMART CONTRACT ARCHITECTURE

2.1 Six Core Programs — Overview

All SAEP programs are written in Rust using the Anchor 0.30+ framework. Each program is independently deployable and upgradeable via its own upgrade authority (held in Squads multisig).

Program	Program ID Seed	Dependencies	CU Budget (per instruction)
AgentRegistry	SAEPreg1111...	CapabilityRegistry, Token-2022	~50K CUs register; ~20K update; ~15K query
TreasuryStandard	SAEPtre1111...	Jupiter CPI, Token-2022, AgentRegistry	~80K CUs fund; ~60K stream-init; ~40K withdraw
TaskMarket	SAEPtsk1111...	TreasuryStandard, ProofVerifier, SolRep oracle	~120K CUs create; ~80K submit; ~400K verify+settle
DisputeArbitration	SAEPdsp1111...	TaskMarket, TreasuryStandard	~60K CUs raise; ~80K arbitrate; ~50K slash
GovernanceProgram	SAEPgov1111...	NXSStaking, FeeCollector	~40K CUs propose; ~20K vote; ~80K execute
FeeCollector	SAEPfee1111...	Token-2022 TransferHook, NXSStaking	~60K CUs collect; ~100K distribute; ~30K burn

2.2 AgentRegistry Program — Full Specification

PDA Structure

```
// Agent Identity PDA
seeds = [b"agent", operator_pubkey.as_ref(), agent_id.as_ref()]

pub struct AgentAccount {
    pub operator:         Pubkey,         // controlling wallet
    pub did:              [u8; 32],        // unique DID hash
    pub manifest_uri:     String,          // IPFS/Arweave CID
    pub capability_mask:  u128,            // bitmask of capability tags (supports
128 tags)
    pub price_lamports:   u64,             // base price in SAEP micro-units
```

```
    pub stream_rate:      u64,              // per-second streaming rate (0 =
disabled)
    pub reputation:       ReputationScore, // 6-dim composite score struct
    pub jobs_completed:   u64,
    pub jobs_disputed:    u32,
    pub stake_amount:     u64,              // SAEP tokens staked
    pub status:           AgentStatus,      // Active|Paused|Suspended|Deregistered
    pub version:          u32,              // manifest version counter
    pub registered_at:    i64,
    pub last_active:      i64,
}
```

Key Instructions

- register_agent: Validates capability_mask against CapabilityRegistry approved bitmask, collects SAEP registration stake (minimum enforced by GovernanceCore param), initializes PDA, emits AgentRegistered event for indexer.
- update_manifest: Owner-only. Increments version counter. Validates new capability_mask is not requesting unapproved capabilities. Emits ManifestUpdated event.
- delegate_control: Allows operator to assign a secondary signer for routine agent operations (e.g., pause/resume) without full operator key. Uses multisig threshold check.
- record_job_outcome: Called by TaskMarket on job completion. Atomically increments jobs_completed, updates reputation score via SolRep oracle CPI, updates last_active timestamp.

2.3 TreasuryStandard Program — Full Specification

Agent PDA Wallet Architecture

Each registered agent gets a deterministic PDA wallet that can hold any SPL token. The TreasuryStandard program enforces spending limits at the instruction level — not at the application level.

```
// Treasury PDA
seeds = [b"treasury", agent_did.as_ref()]

pub struct AgentTreasury {
    pub agent_did:         [u8; 32],
    pub operator:          Pubkey,
    pub daily_spend_limit: u64,          // in USDC lamports
    pub per_tx_limit:      u64,
    pub weekly_limit:      u64,
    pub spent_today:       u64,          // resets at UTC midnight
    pub spent_this_week:   u64,          // resets Monday UTC
    pub last_reset_day:    i64,
    pub streaming_active:  bool,
    pub stream_counterparty: Option<Pubkey>,
```

```
    pub stream_rate_per_sec: u64,
}
```

Streaming Payments Architecture

SAEP implements payment streaming via time-locked state channels anchored to Solana's clock. The streaming mechanism uses Solana's native sysvar::clock for rate enforcement:

1. init_stream: Client and agent agree on stream rate (SAEP tokens/second). Client deposits stream_deposit (rate * max_duration) into escrow PDA. Stream start_time recorded.
2. withdraw_earned: Agent calls at any time. Contract computes earned = rate * (current_time - start_time). Transfers earned amount, leaves remainder in escrow. Updates watermark.
3. close_stream: Either party can close. Agent gets earned-to-date. Client gets remaining deposit. Emits StreamClosed event.
4. Jupiter auto-swap: If agent needs USDC but client paid in SOL, TreasuryStandard CPIs to Jupiter to swap atomically at settlement. No separate transaction needed.

2.4 TaskMarket Program — Full Specification

Task Contract State Machine

```
Created -> Funded -> InExecution -> ProofSubmitted -> Verified -> Released
         \-> Expired (deadline passed, refunded)
                                    \-> Disputed -> ArbitrationPending -> Resolved
```

Task Contract PDA Structure

```
seeds = [b"task", client_pubkey.as_ref(), task_nonce.as_ref()]

pub struct TaskContract {
    pub task_id:            [u8; 32],
    pub client:             Pubkey,
    pub agent_did:          [u8; 32],
    pub payment_mint:       Pubkey,       // USDC, SAEP, or approved token
    pub payment_amount:     u64,
    pub protocol_fee:       u64,          // 0.1% to FeeCollector
    pub solrep_fee:         u64,          // 0.05% to reputation pool
    pub task_hash:          [u8; 32],     // keccak256 of task description
    pub result_hash:        [u8; 32],     // set on submission
    pub proof_key:          [u8; 32],     // Groth16 proof reference
    pub milestone_count:    u8,           // 0 = single payment
    pub milestones_complete:u8,
    pub status:             TaskStatus,
    pub created_at:         i64,
    pub deadline:           i64,
    pub dispute_window_end: i64,          // deadline + 86400 (24h)
```

```
}
```

Proof of Completion — Groth16 Integration

SAEP uses a Groth16 proof system via Light Protocol for cryptographically verifiable task completion. The circuit proves: (1) the agent possesses the result matching task_hash, (2) the result meets declared success criteria, (3) the result was produced within the task deadline.

- Circuit design (Circom 2.0): Three signals — task_hash_preimage, result_hash, success_criteria_satisfied (boolean). Public inputs: task_hash, result_hash. Private input: actual result content.
- Proof generation: Off-chain via SAEP's Proof Generation Service. Agents submit results to the service, which generates the Groth16 proof and submits it on-chain on behalf of the agent.
- On-chain verification: ProofVerifier program loads verifying key from GovernanceCore-controlled VerifierKey PDA. Runs bn254 pairing check via Light Protocol. ~400K CUs under current limits; ~200K CUs post-SIMD-0334.
- Batch verification: For agents completing many low-value tasks, ProofVerifier.batch_verify accepts up to 10 proofs in a single transaction using recursive proof aggregation. Reduces per-proof CU cost to ~80K.

2.5 DisputeArbitration Program

Staked Arbitrator System

Disputes are resolved by a pool of staked arbitrators selected via verifiable random function (VRF). SAEP uses Switchboard VRF for provably fair arbitrator selection.

- Arbitrator registration: Any SAEP holder can register as arbitrator by staking minimum 10,000 SAEP tokens. Stake is slashed for dishonest arbitration.
- Selection: On dispute raised, Switchboard VRF selects 3 arbitrators from the qualified pool weighted by stake. Selection is verifiable on-chain — no manipulation possible.
- Multi-round arbitration: Round 1 = 3 arbitrators, 48-hour window. If 2-of-3 disagree, Round 2 escalation = 5 arbitrators, 72-hour window. Appeals cost additional collateral.
- Slashing mechanics: Arbitrators who vote with the losing minority AND whose vote is flagged as bad-faith (repeated pattern, same-side bias) have 10% of stake slashed per incident.
- Resolution execution: On majority decision, DisputeArbitration CPIs TaskMarket to execute: release to agent (agent wins), refund to client (client wins), or partial split.

2.6 Deployment & Upgrade Architecture

Program	Upgrade Authority	Upgrade Process	Timelock
AgentRegistry	Squads 4-of-7 multisig	Governance proposal +	7 days

		multisig sign	
TreasuryStandard	Squads 4-of-7 multisig	Governance proposal + multisig sign	7 days
TaskMarket	Squads 4-of-7 multisig	Governance proposal + multisig sign	14 days (critical path)
DisputeArbitration	Squads 4-of-7 multisig	Governance proposal + multisig sign	14 days (critical path)
GovernanceProgram	Squads 6-of-9 multisig	Meta-governance proposal required	21 days
FeeCollector	Squads 4-of-7 multisig	Governance proposal + multisig sign	7 days
ProofVerifier	Squads 4-of-7 multisig	Verifier key rotations via governance	7 days

PART III: OFF-CHAIN INFRASTRUCTURE

3.1 Full Infrastructure Stack

Service	Technology	Version	Hosted On	Purpose
IACP Message Bus	Redis Streams + custom WS server	Redis 7.x	AWS ElastiCache + EC2	Real-time agent-to-agent messaging. Off-chain routing, on-chain anchoring of critical messages
Agent Indexer	Custom Rust indexer (Yellowstone)	latest	AWS EC2 (c6i.4xlarge)	Real-time PDA state sync to Postgres. Powers agent discovery API queries
Proof Generation SVC	Node.js + snarkjs + Circom	Node 20.x	AWS ECS (GPU-enabled)	Groth16 proof generation as a service. Accepts task results, outputs proofs for on-chain submission
Oracle Network	Switchboard V2 + Pyth	latest	Decentralized	Price feeds (SAEP/USDC/SOL), compute cost oracles, VRF for dispute arbitrator selection
Discovery API	NestJS + Fastify	NestJS 11.x	Vercel Edge Functions	REST + WebSocket API for agent search. Queries Postgres index. Real-time subscription via WS
Database	Supabase (Postgres 16)	PG 16	Supabase Cloud	Agent metadata, job history, analytics events, reputation history
Cache Layer	Redis 7 (Upstash)	Redis 7.x	Upstash (edge)	Discovery query results, agent manifests, price feed cache. TTL = 400ms (1 Alpenglow slot)

File Storage	IPFS via Pinata + Arweave	latest	Pinata / Arweave network	Agent capability manifest storage. IPFS for mutable manifests; Arweave for permanent archival
Monitoring	Grafana + Prometheus + Loki	latest	AWS EC2	Program health, indexer lag, proof gen queue depth, RPC error rates
Alerting	PagerDuty + Slack webhooks	latest	Cloud	On-call alerts for: program errors, indexer lag >2 slots, proof queue overflow, escrow anomalies

3.2 IACP Message Bus — Deep Specification

The Inter-Agent Communication Protocol (IACP) is SAEP's real-time messaging layer. It handles all agent-to-agent communication, with on-chain anchoring for financial-critical messages.

Message Types & Routing

Message Type	Routing	On-Chain Anchoring	TTL
TaskRequest	Direct (requester → agent DID)	No — off-chain only	5 minutes
TaskAccept	Direct (agent → requester)	Yes — TaskMarket.create_task CPI	N/A (permanent)
TaskReject	Direct (agent → requester)	No	Ephemeral
StatusUpdate	Direct (agent → requester)	No	30 seconds
ResultDelivery	Direct (agent → requester)	Yes — TaskMarket.submit_result	N/A (permanent)
PaymentRequest	Direct (agent → requester)	Yes — TreasuryStandard CPI	N/A (permanent)
Broadcast RFP	Pub/sub to capability channel	No	15 minutes
Dispute	Direct (client → arbitration prog)	Yes — DisputeArbitration CPI	N/A (permanent)

A2A Orchestration	Direct (orchestrator → sub-agent)	Partial — result anchored only	Per-task TTL

Message Bus Implementation

The IACP message bus uses Redis Streams as the transport layer with a custom WebSocket server for real-time delivery. Architecture:

5. Agents maintain persistent WebSocket connections to IACP WS server authenticated via SIWS session
6. Outbound messages written to Redis Stream keyed by recipient DID: saep:iacp:{recipient_did}
7. WS server maintains connection pool; delivers messages to connected agent, buffers for offline agents
8. Financial-critical messages (TaskAccept, ResultDelivery, Payment) trigger automatic on-chain anchoring via a worker pool that CPIs the relevant SAEP program
9. Worker pool uses Jito bundles for atomic message anchoring + state transition in single block
10. All messages encrypted using recipient's Ed25519 public key (derived from DID). Operator can audit own agents; cannot read other agents' messages

3.3 Agent Indexer — Yellowstone gRPC

The SAEP indexer consumes Yellowstone gRPC slot streams from Helius, processes account updates for all SAEP program PDAs, and maintains a real-time Postgres replica for fast API queries.

Indexer Architecture

11. Yellowstone gRPC subscription on Helius dedicated node — filtered to SAEP program account updates only
12. Account update events decoded using SAEP IDL (Anchor account discriminator matching)
13. Decoded state diffed against current Postgres record — only changed fields written
14. Events emitted to Redis Pub/Sub for live frontend subscriptions (agent status, task state, fee burns)
15. Postgres schema maintains full history: every state transition stored with slot number and timestamp
16. Indexer lag metric tracked: alert fires if indexer falls >2 slots behind chain tip

Indexer Performance Targets

- Processing latency: < 50ms from account update to Postgres write
- Throughput: sustained 10,000 account updates/second (headroom for 1B txn/year target)

- Reorg handling: Solana's probabilistic finality before Alpenglow means indexer must handle slot rollbacks. All writes include slot number; rollback procedure re-processes from last finalized slot
- Post-Alpenglow optimization: Once Alpenglow mainnet, sub-150ms finality eliminates most rollback scenarios. Indexer can be simplified to treat first confirmation as final

3.4 Proof Generation Service

The Proof Generation Service (PGS) is a critical off-chain component that generates Groth16 proofs on behalf of agents. Agents submit task results; PGS returns a proof ready for on-chain submission.

Component	Technology	Purpose
API Layer	NestJS + Fastify	Receives result submissions from agents. Returns proof + verification tx.
Proof Worker Pool	Bull queues + GPU workers	Parallel Groth16 proof generation. Queue depth monitored; auto-scales on AWS ECS.
Circom Circuit	Circom 2.0 + snarkjs	Task completion circuit. Generates bn254 elliptic curve proofs compatible with Light Protocol verifier.
Trusted Setup	Hermez/Zcash Powers of Tau	Phase 1 trusted setup from existing ceremony. Phase 2 SAEP-specific setup with multi-party ceremony pre-launch.
Proof Submission	Helius staked RPC	Signed by protocol fee payer wallet. Submits ProofVerifier.verify_proof instruction. Returns tx signature to agent.
Batch Aggregation	Nova recursive SNARKs (Phase 3)	Aggregate up to 10 proofs into single recursive proof. Reduces on-chain verification CU cost by 75%.

PART IV: DEVELOPMENT ENVIRONMENT & DEPLOYMENT

4.1 Local Development Setup

Prerequisites

```
# Solana toolchain (target: 1.18+, Firedancer-compatible)
sh -c "$(curl -sSfL https://release.solana.com/stable/install)"

# Anchor AVM
cargo install --git https://github.com/coral-xyz/anchor avm --locked
avm install 0.30.1 && avm use 0.30.1

# Node.js + pnpm
nvm install 20 && nvm use 20 && npm install -g pnpm

# SAEP monorepo
git clone git@github.com:saep-protocol/saep && cd saep
pnpm install

# Local validator with SAEP programs
anchor localnet  # spins up solana-test-validator + deploys all 6 programs
```

Environment Variables

```
# .env.local
HELIUS_API_KEY=xxx                      # RPC + Yellowstone access
HELIUS_STAKED_URL=https://...           # Priority staked connection
YELLOWSTONE_ENDPOINT=grpc.helius.dev   # gRPC slot streaming
REDIS_URL=redis://localhost:6379        # IACP message bus
DATABASE_URL=postgresql://...           # Supabase connection
PROOF_SERVICE_URL=http://localhost:3001 # Local PGS instance
JITO_BLOCK_ENGINE=bundles.jito.wtf      # Jito bundle endpoint
SWITCHBOARD_VRF_KEY=xxx                 # Arbitrator selection VRF
```

4.2 Anchor.toml Configuration

```
[programs.devnet]
agent_registry      = "SAEPreg11111111111111111111111111111111111111"
treasury_standard   = "SAEPtre11111111111111111111111111111111111111"
task_market         = "SAEPtsk11111111111111111111111111111111111111"
dispute_arbitration = "SAEPdsp11111111111111111111111111111111111111"
governance_program  = "SAEPgov11111111111111111111111111111111111111"
fee_collector       = "SAEPfee11111111111111111111111111111111111111"
```

```
[test]
startup_wait = 5000

[provider]
cluster  = "devnet"
wallet   = "./wallets/saep-deploy.json"
```

4.3 Contract Deployment Order

17. CapabilityRegistry — seed with initial 32 capability tags (bitmask positions 0-31)
18. GovernanceCore — deploy with initial multisig members, default parameters, 48h devnet timelock
19. FeeCollector — initialize with burn_bps=1000 (10%), staker_share_bps=5000 (50%), grant_bps=2000 (20%)
20. AgentRegistry — initialize with GovernanceCore and CapabilityRegistry addresses
21. TreasuryStandard — initialize with AgentRegistry, Jupiter program ID, approved payment mints [USDC, SAEP]
22. ProofVerifier — initialize with Groth16 verifying key, Light Protocol verifier program ID
23. TaskMarket — initialize with all program addresses; set fee rates; connect SolRep oracle
24. DisputeArbitration — initialize with TaskMarket, Switchboard VRF program, arbitrator min stake

4.4 CI/CD Pipeline

Stage	Tool	Triggers	Action
Lint	cargo clippy + ESLint	Every PR	Rust: zero warnings policy. TS: strict mode. Fails PR on any warning.
Unit Tests	cargo test + Vitest	Every PR	All Anchor program unit tests. SDK-UI hook tests. 95% coverage gate.
Integration Tests	Anchor.toml test suite	Every PR	Full program interaction tests on localnet. All 6 programs + cross-program calls.
Devnet Deploy	anchor deploy + shell scripts	Merge to develop	Auto-deploys all programs to devnet. Runs smoke test suite (Playwright).
Security Scan	cargo-audit + semgrep	Weekly + pre-audit	Rust dependency CVE scan. Semgrep rules for common Solana vulnerabilities.
Mainnet Deploy	Manual + multisig sign	Tag v*.*.* release	Program binary built in reproducible build container. 4-of-7 multisig approves BPF deploy.

PART V: SECURITY ARCHITECTURE

5.1 Smart Contract Security Checklist

Every SAEP program must pass this checklist before mainnet deployment. Verified by OtterSec, Neodyme, and Halborn across three sequential audits.

Category	Checks Required
Account Validation	Owner checks on all accounts; signer checks; PDA derivation matches expected seeds; discriminator validation on all Anchor accounts
Re-entrancy	No CPI calls within escrow release paths that could trigger re-entrant execution; state written before CPI calls
Integer Safety	All u64 arithmetic uses checked_add/checked_mul; no direct subtraction without underflow check; BN bounds validated
Authorization	Every mutable instruction: operator-only or program-authority checks; no missing signer validations; emergency pause respected
Oracle Safety	All Pyth/Switchboard price reads validated: price age < 60s, confidence interval < 1%, status == Trading
Slashing Safety	Slashing conditions time-locked 30 days before activation; slash amount bounded by min(stake, max_slash_cap); appeal window enforced
Upgrade Safety	All upgrade authorities in Squads multisig from day 1; no program-controlled upgrade authorities; timelock enforced on-chain
Token Safety	Token-2022 extension combinations validated (TransferHook + ConfidentialTransfer incompatibility acknowledged and avoided); fee calculations integer-safe

5.2 Operational Security

- All signing keys in hardware security modules (HSMs) for protocol operations. No hot wallets for admin operations.
- Squads multisig for all upgrade authorities — 4-of-7 for standard programs, 6-of-9 for GovernanceCore. Signers geographically distributed.
- Bug bounty live at mainnet launch: $50K for critical (escrow bypass, unauthorized slashing); $10K for high (unauthorized agent control); $2K for medium.
- Incident response runbook: defined procedures for exploit detection (Helius anomaly alerts), emergency pause execution (multisig path), disclosure timeline, and post-mortem process.
- Proof system trusted setup: Multi-party computation ceremony with minimum 20 participants, including independent cryptographers. Ceremony transcript published publicly.

5.3 Post-Audit Upgrade Schedule

Milestone	Audit	Scope	Timeline
Alpha devnet	OtterSec — Audit 1	AgentRegistry + TreasuryStandard + TaskMarket	Month 4
Alpha mainnet	Neodyme — Audit 2	DisputeArbitration + GovernanceCore + FeeCollector	Month 7
Token launch	Halborn — Audit 3	Full protocol re-audit + Token-2022 extensions + IACP	Month 9
Phase 2 upgrade	OtterSec — Audit 4	A2A streaming protocol + Phase 2 feature additions	Month 14
Confidential ext.	Neodyme — Audit 5	Confidential transfers integration (when ZK ElGamal live)	Month 22+

PART VI: OPTIMIZATION & UPGRADE ROADMAP

> *This section documents post-launch optimization opportunities identified during architecture review. These are improvements SAEP should implement after initial mainnet deployment, ordered by impact-to-complexity ratio.*

6.1 Immediate Optimizations (Month 1-3 Post-Launch)

Optimization	Impact	Complexity	Description
Compute Unit auto-tuning	High	Low	Implement dynamic CU budgeting: simulate every tx, set limit to result + 15% buffer. Reduces overpaying by avg 30%. Saves users SOL on every interaction.
Account compression for badges	High	Low	Migrate agent reputation badges and credential records to Light Protocol ZK compression. Reduces per-badge rent from ~0.002 SOL to near-zero. Critical for scale.
Proof batch verification	High	Medium	Implement ProofVerifier.batch_verify for up to 10 Groth16 proofs per tx using recursive aggregation. Reduces per-proof cost from 400K to ~80K CUs. Enables high-volume micro-task market.
Redis Streams backpressure	Medium	Low	Add consumer group acknowledgment to IACP Redis Streams. Currently fire-and-forget. Adds delivery guarantee for financial-critical messages without full queue re-architecture.
Address Lookup Table caching	Medium	Low	Pre-build and cache ALTs for common account combinations (task creation, dispute, governance vote). Reduces per-tx size by ~200 bytes. Prevents 1232-byte limit issues at scale.

6.2 Phase 2 Upgrades (Month 6-12)

Upgrade	Unlocks	Depends On	Description
Nova recursive proofs	Infinite agent pipeline depth	Nova proving system maturity	Replace Groth16 with Nova for recursive SNARK aggregation. Enables arbitrarily deep A2A

			pipelines provable in O(1) proof size regardless of pipeline depth.
SIMD-0334 integration	50% CU reduction on ZK verify	SIMD-0334 mainnet activation	Upgrade ProofVerifier program to use new ZK syscalls post-SIMD-0334. No user-facing change but significant fee reduction.
Confidential escrow (Phase 3)	Enterprise privacy for task amounts	ZK ElGamal audit completion	Once ZK ElGamal Program audit completes, add optional confidential escrow: client and agent agree on encrypted payment amount. Auditor key assigned for KYC/AML compliance.
Cross-chain agent bridging	Ethereum/Base agents on SAEP	Wormhole / LayerZero integration	SAEP agent DIDs bridgeable to EVM chains. Ethereum-native agents can register via bridge, accept USDC tasks from Solana clients, settle cross-chain atomically.
Decentralized indexer	Protocol censorship resistance	Solana Foundation indexer grants	Replace centralized Yellowstone indexer with permissionless indexer network. Multiple independent indexers; SAEP Discovery API queries any available indexer.

6.3 Long-Term Protocol Upgrades (Year 2+)

- Alpenglow-native settlement: Once Alpenglow mainnet, remove all optimistic settlement logic from TaskMarket. Treat first confirmation as final. Removes 24h dispute window for Alpenglow-confirmed tasks (instant finality = instant settlement).
- Firedancer tile integration: As Firedancer captures 50%+ stake, SAEP can publish a Firedancer tile plugin for specialized SAEP transaction processing. Dedicated tile for proof verification = near-zero CU cost for SAEP operations on Firedancer validators.
- IoT agent support: Add AgentRegistry capability category for IoT devices. Physical devices register as SAEP agents, receive micropayments for sensor data, pay each other for data access. Enabled by Alpenglow's <150ms finality making sub-cent IoT micropayments economically viable.
- SVM execution environments: As Solana's SVM (Solana Virtual Machine) gets extracted for use in other chains (Eclipse, SoonSVM, etc.), SAEP programs can be deployed on SVM L2s with the same DIDs and reputation scores. Protocol-level cross-SVM identity.
- MCP (Model Context Protocol) native integration: Implement SAEP as a native MCP server. Any Claude/GPT/Gemini agent with MCP support can discover and hire SAEP agents directly from within the LLM context window without writing a single line of code.